HA TRADING LAB · MASTERY CHECKLIST

Master Every Component

A step-by-step checklist to understand, operate, break, read and change every part of the HA Trading Lab, from HAProxy to Kafka, RabbitMQ, Redis and PostgreSQL, until you can guarantee the code.

12 checkpoints
Owning the lab, then one per component, each with five levels to tick.

Predict, then break
Every failure drill is written down first, then checked on the live dashboard.

Signed off
Each checkpoint is passed without notes and checked by a reviewer.

1 How to use this checklist
2 Checkpoint 0: own the lab
3 Component checkpoints (1–11)
4 Final checkpoint

1 How to use this checklist

Every component has **five levels**. A component is mastered when all five are passed **without notes**, and someone else has seen it (sign the checkpoint). When every checkpoint and the final one are signed, you can stand behind the code.

1 · Explain
Say what it does and why, in your own words.

2 · Inspect
See its live state with a command and read it correctly.

3 · Break
Kill it and predict the result before you do.

4 · Read
Walk through its code and config line by line.

5 · Change
Make a small change, deploy, prove it, revert.

Before every Break step: untick **Auto-heal** on the dashboard (otherwise it restarts your victim after 15 s). Run commands in Ubuntu from `/mnt/d/HA_Trading_Lab`. Everything stays inside the lab: the exchange is simulated, nothing touches IBKR or real money. Every inspect command in this list was tested on the running lab. After a Break step, bring everything back with **Restore everything** on the dashboard or `docker compose up -d`.

Four-week plan

Week 1	Checkpoints 0–2
Week 2	Checkpoints 3–6
Week 3	Checkpoints 7–9
Week 4	Checkpoints 10–11 + Final

Progress tracker

Checkpoint	1 · Explain	2 · Inspect	3 · Break	4 · Read	5 · Change	Date	Signed off
0 · Own the lab	☐	☐	☐	☐	☐		
1 · HAProxy	☐	☐	☐	☐	☐		
2 · API servers	☐	☐	☐	☐	☐		
3 · PostgreSQL + Patroni + etcd	☐	☐	☐	☐	☐		
4 · PgBouncer	☐	☐	☐	☐	☐		
5 · Outbox + Relay	☐	☐	☐	☐	☐		
6 · Kafka	☐	☐	☐	☐	☐		
7 · Order router	☐	☐	☐	☐	☐		
8 · Feed + price lines	☐	☐	☐	☐	☐		
9 · Bridge → RabbitMQ → Worker	☐	☐	☐	☐	☐		
10 · Redis + Sentinel	☐	☐	☐	☐	☐		
11 · Control, audit and dashboard	☐	☐	☐	☐	☐		
★ · Final: guarantee			☐ all 7 items				

2 Checkpoint 0: own the lab

- [ ] `git init && git add -A && git commit -m "baseline"` : every experiment can be undone with `git checkout .`
- [ ] Start / stop / status: `docker compose up -d` · `docker compose down` · `docker compose ps` (38 running)
- [ ] Logs: `docker compose logs -f router-1` · `docker logs --since 5m hal-control`
- [ ] Deploy one service: `docker compose up -d --build --no-deps control`
- [ ] Read *How Every Part Works* once, end to end

Pass: Restart the whole lab from zero and reach "audit all 0" without help. Passed on _____ · checked by _____

3 Component checkpoints

1 HAProxy WEEK 1

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Why round robin?
  - What do `fall 2 rise 2` mean?
  - What do `retries` and `redispatch` do?
- [ ] **2 · Inspect**

```
docker exec hal-haproxy-1 sh -c "wget -qO- 'http://127.0.0.1:8404/stats;csv' | cut -d, -f1,2,18"
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker stop hal-api-2
```

→ Expect: marked DOWN within ~2 s; traders unaffected; audit 0

- [ ] **4 · Read**
  - `config/haproxy-edge.cfg (every line)`
- [ ] **5 · Change** Set `inter 1s` → `inter 3s`, redeploy, measure the slower detection, revert. Audit must stay 0; revert with git.

2 API servers WEEK 1

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Why does ONE transaction write the order and the outbox?
  - What does `client_ref` protect against?
  - Why is the login token stateless?
- [ ] **2 · Inspect**

```
docker logs --since 2m hal-api-1
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-api-3
```

→ Expect: a few trader errors in the last 10 s, all recovered by retry; lost = 0

- [ ] **4 · Read**
  - `app/api.py: place_order, cancel_order, make_token / verify_token, best_effort, tick_loop, main (drain)`
- [ ] **5 · Change** Lower `STALE_AFTER` in app/common.py from 3 s to 1 s; watch "refused (stale)" rise; revert. Audit must stay 0; revert with git.

3 PostgreSQL + Patroni + etcd WEEK 2

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Why quorum commit?
  - Why a 20 s leader-key TTL?
  - Why refuse to run with one node left?
  - What does pg_rewind do?

- [ ] **2 · Inspect**

```
docker exec hal-pg-1 curl -s localhost:8008/cluster
```

```
docker exec hal-etcd-1 etcdctl endpoint status --cluster -w table
```

```
docker exec -it hal-pg-1 psql -U postgres -d trading      (explore every table in app/schema.sql)
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-pg-1        # use the node the inspect command shows as leader
```

→ Expect: ~20 s of "try again", then a new leader; lost = 0

```
docker stop hal-pg-2 hal-pg-3
```

→ Expect: DATABASE DOWN and NO promotion: explain why

```
docker kill hal-etcd-3      # use the endpoint with IS LEADER = true
```

→ Expect: new etcd leader in ~2 s; the database keeps working

- [ ] **4 · Read**
  - `images/postgres/patroni.yml (every line)`
  - `images/postgres/post_bootstrap.sh`
  - `images/postgres/pgbackrest.conf`

- [ ] **5 · Change** Restore drill: restore the latest backup into a spare test container (never the live cluster) and state the RPO. Audit must stay 0; revert with git.

4 PgBouncer WEEK 2

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Transaction vs session pooling
  - Why prepared statements are disabled
  - How the built-in HAProxy follows the leader

- [ ] **2 · Inspect**

```
docker exec -e PGPASSWORD=apppass hal-pg-1 psql -h pgbouncer-1 -p 6432 -U app pgbouncer -c "show pools"
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker stop hal-pgbouncer-1
```

→ Expect: apps switch to pgbouncer-2; nothing noticed

- [ ] **4 · Read**
  - `images/pgbouncer/pgbouncer.ini`
  - `images/pgbouncer/haproxy-pg.cfg`
  - `app/common.py: dsn()`

- [ ] **5 · Change** Set `default_pool_size` 15 → 3; watch `cl_waiting` rise under load; revert. Audit must stay 0; revert with git.

5 Outbox + Relay WEEK 2

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - What breaks if the API publishes to Kafka directly?
  - Why FOR UPDATE SKIP LOCKED?
  - Why can a message be sent twice, and why is that fine?
- [ ] **2 · Inspect**

```
docker exec hal-pg-1 psql -U postgres -d trading -c "select count(*) filter (where published_at is null) unsent, count(*) total from outbox"
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker stop hal-relay-1 hal-relay-2   # 30 s later: docker start hal-relay-1 hal-relay-2
```

→ Expect: unsent grows, then drains to ~0; nothing lost

- [ ] **4 · Read**
  - `app/relay.py (whole file)`
- [ ] **5 · Change** Write an outbox cleanup job: delete published rows older than 1 day (the table is 728 MB today). Audit must stay 0; revert with git.

6 Kafka WEEK 2

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Partition, leader, ISR, consumer group, offset
  - acks=all + min.insync.replicas=2
  - Why are the price lines single-copy on purpose?
- [ ] **2 · Inspect**

```
docker exec hal-kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic orders.submit
```

```
docker exec hal-kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group notify-bridge
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-kafka-1     # the broker leading the most partitions
```

→ Expect: new leaders in 0.5–3 s; price gap < 0.2 s

```
docker stop hal-kafka-1 hal-kafka-2
```

→ Expect: producers stop (ISR < 2), outbox backlog grows, nothing lost

- [ ] **4 · Read**
  - `docker-compose.yml (Kafka environment)`
  - `app/common.py: producer_conf, TICK_TOPICS`
  - `app/control.py: bootstrap (topic creation)`
- [ ] **5 · Change** Create a test topic (3 partitions, RF 3), produce to it and read it back with the console consumer. Audit must stay 0; revert with git.

7 Order router WEEK 3

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Lease, epoch, fencing
  - Why the router stops itself after 2 s without renewing
  - What reconcile does
  - Why the fill id is a database key
- [ ] **2 · Inspect**

```
docker exec hal-pg-1 psql -U postgres -d trading -c "select * from router_lease" -c "select * from router_state"
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-router-1    # the holder shown in router_lease
```

→ Expect: epoch +1, takeover in ~3–4 s, reconcile runs; sent twice = 0

```
docker pause hal-router-1; sleep 10; docker unpause hal-router-1
```

→ Expect: the old router wakes up fenced; its writes are rejected (split-brain test)

- [ ] **4 · Read**
  - `app/router.py: lease_loop, active_epoch, fenced, send, claim, sweep, reconcile, apply, exec_loop, END_OF_SESSION (read twice)`
- [ ] **5 · Change** Log every write that fenced() rejects; repeat the pause test and find the line in the logs. Audit must stay 0; revert with git.

8 Feed + price lines WEEK 3

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Why two independent lines?
  - Why (boot, seq)?
  - What resync() fixes
- [ ] **2 · Inspect**

```
Dashboard: Prices panel (tick gap)
```

```
docker logs --since 1m hal-feed-1
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker stop hal-feed-1 && docker restart hal-exchange
```

→ Expect: prices continue on line B; both feeds resync

- [ ] **4 · Read**
  - `app/feed.py`
  - `app/api.py: tick_loop`
- [ ] **5 · Change** Show the per-line tick rate on the dashboard's Prices panel. Audit must stay 0; revert with git.

9 Bridge → RabbitMQ → Worker WEEK 3

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Kafka vs RabbitMQ
  - Publisher confirms and manual ack
  - Quorum queue, dead-letter queue, pause_minority
- [ ] **2 · Inspect**

```
docker exec hal-rabbit-1 rabbitmqctl -q list_queues name type leader members messages consumers
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-rabbit-3    # the leader shown by list_queues
```

→ Expect: leader moves in < 1 s; notifications missing = 0

```
docker stop hal-worker-1 hal-worker-2   # 30 s later: docker start hal-worker-1 hal-worker-2
```

→ Expect: messages grows, then drains when they return

- [ ] **4 · Read**
  - `app/bridge.py`
  - `app/worker.py`
  - `config/rabbitmq.conf`
  - `app/control.py: bootstrap (RabbitMQ part)`
- [ ] **5 · Change** Publish one malformed message by hand and watch it reach notifications.dlq after 5 tries. Audit must stay 0; revert with git.

10 Redis + Sentinel WEEK 4

Passed on _____ · checked by _____

- [ ] **1 · Explain** Answer in your own words:
  - Why Redis may fail without stopping trading
  - Sentinel quorum 2, down-after 3000 ms
  - AOF persistence
- [ ] **2 · Inspect**

```
docker exec hal-sentinel-1 redis-cli -p 26379 sentinel get-master-addr-by-name lab
```

```
docker exec hal-redis-1 redis-cli role
```

- [ ] **3 · Break** Write your prediction **before** you run it:

```
docker kill hal-redis-3     # the master Sentinel reports
```

→ Expect: new master in 4–5 s

```
docker stop hal-redis-1 hal-redis-2 hal-redis-3
```

→ Expect: trading continues; the circuit breaker skips Redis

- [ ] **4 · Read**
  - `config/redis-entry.sh`
  - `config/sentinel-entry.sh`
  - `app/api.py: best_effort, rate_limited`
- [ ] **5 · Change** Make Sentinel's down-after 5 s, redeploy, and re-measure the failover time; revert. Audit must stay 0; revert with git.

11 Control, audit and dashboard WEEK 4

Passed on _____ · checked by _____

- [ ] **1 · Explain** — Answer in your own words:
  - Every audit check: what it compares and why it proves "nothing lost"
  - Why the trader-errors tile returns to 0 after 10 s
- [ ] **2 · Inspect**

```
curl -s http://127.0.0.1:9700/api/state      (the audit and trader counters)
```

- [ ] **3 · Break** — Write your prediction **before** you run it:

```
docker stop hal-control
```

  → Expect: the dashboard goes away; trading and failover carry on

- [ ] **4 · Read**
  - `app/control.py: Trader, audit_once, detect_events, chaos, autoheal_loop, backup_loop`
  - `app/ui/index.html: tiles()`
- [ ] **5 · Change** — Add an audit check "no order filled beyond its quantity" as a dashboard tile. Audit must stay 0; revert with git.

4 Final checkpoint: you can guarantee it

Pass all seven, without notes, in front of a reviewer.

- ☐ **Trace one real order** end to end: its rows in orders, outbox, fills and notifications, its Kafka message and its exchange fill; explain every timestamp (guide, section 4).
- ☐ **Run every drill** in the guide's test catalogue (section 7.4): write the prediction first, then compare. All audits 0.
- ☐ **Take a whole zone down** and back up; explain every line of the event timeline.
- ☐ **Rebuild from nothing**: `docker compose down -v`, start again, reach "audit all 0". **Warning:** `-v` deletes every volume, including the database and the backups: do it last, after the restore drill.
- ☐ **Restore drill**: restore from a backup; state the RPO and RTO you measured.
- ☐ **Explain the limits**: what the lab does not prove yet (real latency, network cuts, disk loss, 10× load).
- ☐ **Teach it**: walk someone through *How Every Part Works* and demo a database failover live.

Final sign-off	Passed on _____	Reviewer _____

Reference documents

- `D:\HA_Trading_Lab\docs\HA_Trading_Lab_Guide.pdf`: the complete guide (architecture, one order end to end, failures, testing, plan to production)
- `D:\HA_Trading_Lab\docs\HA_Lab_How_Every_Part_Works.pdf`: one card per component
- Dashboard: `http://127.0.0.1:9700`